Repository Audit Report

Repository	dipak2-8/dummy-audit-test
Scan	Full Repository Scan
Risk Level	**HIGH**

Repository Audit Report — dipak2-8/dummy-audit-test

Repository Overview

- Total security issues found: 9
- Total quality issues found: 1
- GitHub Actions vulnerabilities: 0
- Open PRs analyzed: 0

Repository Issues

Security Issues

- File: `api.py` | Function: `hash_password` | Line: 14

```
def hash_password(password):
# weak hashing algorithm
return hashlib.md5(password.encode()).hexdigest() # <- ISSUE HERE
```

- Why it is dangerous: MD5 is not a secure password hashing algorithm and can be cracked quickly.
- Fixed version of the full function:

```
def hash_password(password):
```

```
# use a strong password hashing algorithm
return hashlib.scrypt(password.encode(), salt=b"static_salt", n=16384, r=8,
p=1).hex()
```

• Documentation link: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html

---

• File: `api.py` | Function: `fetch_data` | Line: 18

```
def fetch_data(url):
# SSL verification disabled
response = requests.get(url, verify=False) # <- ISSUE HERE
return response.json()
```

• Why it is dangerous: Disabling certificate verification allows insecure TLS connections and enables man-in-the-middle attacks.

• Fixed version of the full function:

```
def fetch_data(url):
response = requests.get(url, timeout=10)
return response.json()
```

• Documentation link: https://bandit.readthedocs.io/en/1.9.4/plugins/b501_request_with_no_cert_validation.html

---

• File: `api.py` | Function: `fetch_data` | Line: 18

```
def fetch_data(url):
# SSL verification disabled
response = requests.get(url, verify=False) # <- ISSUE HERE
return response.json()
```

• Why it is dangerous: The request has no timeout, which can cause the application to hang indefinitely on network issues.

• Fixed version of the full function:

```
def fetch_data(url):
response = requests.get(url, timeout=10)
return response.json()
```

• Documentation link: https://bandit.readthedocs.io/en/1.9.4/plugins/b113_request_without_timeout.html

---

• File: `app.py` | Function: `login` | Line: 6

```
def login(username, password):
# weak hashing
hashed = hashlib.md5(password.encode()).hexdigest() # <- ISSUE HERE
conn = sqlite3.connect("users.db")
cursor = conn.cursor()
# SQL injection again
cursor.execute("SELECT * FROM users WHERE username = '" + username + "'")
return cursor.fetchone()
```

• Why it is dangerous: MD5 is a weak password hashing algorithm and is unsuitable for protecting credentials.

• Fixed version of the full function:

```
def login(username, password):
hashed = hashlib.scrypt(password.encode(), salt=b"static_salt", n=16384, r=8,
p=1).hex()

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

cursor.execute("SELECT * FROM users WHERE username = ?", (username,))

return cursor.fetchone()
```

• Documentation link: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html

---

• File: `app.py` | Function: `login` | Line: 12

```
def login(username, password):
# weak hashing
hashed = hashlib.md5(password.encode()).hexdigest()
conn = sqlite3.connect("users.db")
cursor = conn.cursor()
# SQL injection again
cursor.execute("SELECT * FROM users WHERE username = '" + username + "'") # <-
ISSUE HERE
return cursor.fetchone()
```

• Why it is dangerous: String concatenation with untrusted input can lead to SQL injection.

• Fixed version of the full function:

```
def login(username, password):
hashed = hashlib.scrypt(password.encode(), salt=b"static_salt", n=16384, r=8,
p=1).hex()

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

cursor.execute("SELECT * FROM users WHERE username = ?", (username,))

return cursor.fetchone()
```

• Documentation link:
https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html

---

• File: `database.py` | Function: `get_user` | Line: 10

```
def get_user(user_id):
conn = sqlite3.connect("app.db")
cursor = conn.cursor()
# SQL injection
cursor.execute("SELECT * FROM users WHERE id = " + user_id) # <- ISSUE HERE
return cursor.fetchall()
```

• Why it is dangerous: Concatenating untrusted input into SQL can allow SQL injection.

• Fixed version of the full function:

```
def get_user(user_id):
conn = sqlite3.connect("app.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))
return cursor.fetchall()
```

• Documentation link:
https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html

---

• File: `database.py` | Function: `delete_user` | Line: 17

```
def delete_user(username):
conn = sqlite3.connect("app.db")
```

```
cursor = conn.cursor()
# SQL injection again
query = "DELETE FROM users WHERE username = '" + username + "'" # <- ISSUE HERE
cursor.execute(query)
conn.commit()
```

• Why it is dangerous: Building SQL with string concatenation can allow SQL injection.

• Fixed version of the full function:

```
def delete_user(username):
conn = sqlite3.connect("app.db")
cursor = conn.cursor()
query = "DELETE FROM users WHERE username = ?"
cursor.execute(query, (username,))
conn.commit()
```

• Documentation link: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html

---

• File: `file_handler.py` | Function: `load_config` | Line: 9

```
def load_config(filename):
# arbitrary code execution via pickle
with open(filename, "rb") as f:
return pickle.load(f) # <- ISSUE HERE
```

• Why it is dangerous: Unpickling untrusted data can execute arbitrary code.

• Fixed version of the full function:

```
import json

def load_config(filename):
with open(filename, "r", encoding="utf-8") as f:
return json.load(f)
```

• Documentation link: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle

---

• File: `file_handler.py` | Function: `execute_command` | Line: 19

```
def execute_command(cmd):
# shell injection
```

```
os.system(cmd) # <- ISSUE HERE
```

- Why it is dangerous: Passing untrusted input to a shell can lead to command injection.
- Fixed version of the full function:

```
import subprocess


def execute_command(cmd):
subprocess.run(cmd, check=True, shell=False)
```

- Documentation link: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html

---

Code Quality Issues

- File: `app.py` | Function: `login` | Line: 6

```
def login(username, password):
# weak hashing
hashed = hashlib.md5(password.encode()).hexdigest() # <- ISSUE HERE
conn = sqlite3.connect("users.db")
cursor = conn.cursor()
# SQL injection again
cursor.execute("SELECT * FROM users WHERE username = '" + username + "'")
return cursor.fetchone()
```

- What is wrong: The local variable `hashed` is assigned but never used.
- Fixed version:

```
def login(username, password):
conn = sqlite3.connect("users.db")
cursor = conn.cursor()
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
return cursor.fetchone()
```

- Documentation link: https://docs.astral.sh/ruff/rules/f841

GitHub Actions Vulnerabilities

No GitHub Actions vulnerabilities were found.

Pull Request Analysis

No open pull requests were analyzed.

Overall Risk Assessment

- Risk Level: HIGH
- Reason: The repository contains multiple high-impact security issues, including weak password hashing, SQL injection, insecure TLS usage, unsafe deserialization, and shell command injection.